

Dans ce cours, nous allons simuler le mouvement d'un objet dont l'accélération **change au cours du temps**. Le mouvement ne sera donc pas un MRUA, ce qui fait que vous ne pouvez pas (encore) trouver vous-mêmes les équations du mouvement. Heureusement, la simulation va vous permettre d'obtenir une solution numérique.

Nous simulerons ce mouvement en 1 dimension, puis en 2 dimensions, et nous tracerons les graphiques qui permettent de visualiser ce mouvement.

2 Simuler le mouvement d'un objet avec frottement fluide en 1 dimension

Le mouvement que nous allons étudier est celui d'un objet qui n'est soumis qu'à deux forces : la force de gravité terrestre et le frottement fluide qui est dû à la présence d'air autour de l'objet. C'est donc une situation qui peut modéliser la chute d'une pierre lâchée sans vitesse initiale depuis le haut d'une falaise, ou bien la trajectoire d'une bille lancée à la verticale ou en oblique à l'aide d'un canon.

2.1 Théorie

Force de frottement fluide : Lorsqu'un objet se déplace à vitesse peu élevée dans l'air, alors il est soumis à une force dans le sens opposé de sa vitesse qui s'exprime comme :

$$\vec{F}_f = -K \cdot \vec{v} \quad (1)$$

où $\vec{F}_f$ est la force de frottement fluide en [N], $\vec{v}$ est le vecteur vitesse de l'objet en [m/s] et K est le coefficient qui dépend de la géométrie de l'objet en [N.s/m].

Tir vertical (montée) :

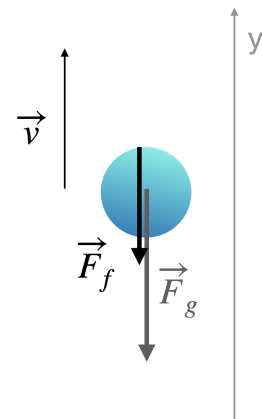
On considère d'abord le cas du tir d'une bille de masse m avec une vitesse initiale verticale. Cela simplifie le problème car il n'y a qu'une seule coordonnée à considérer. On choisit comme axe vertical l'axe y orienté vers le haut avec l'origine au niveau du sol. Pendant la montée de la bille, la vitesse est orientée vers le haut, et la bille est soumise à la force de gravité $\vec{F}_g$ qui est orientée vers le bas, et la force de frottement fluide $\vec{F}_f$, qui est orientée vers le bas puisque $\vec{v}$ est vers le haut.

Nous obtenons l'accélération avec la deuxième loi de Newton appliquée à la bille :

$$m\vec{a} = \vec{F}_g + \vec{F}_f$$

En projetant sur l'axe vertical y , on obtient :

$$ma_y = -mg - kv_y$$



On isole l'accélération :

$$a_y = -g - \frac{K}{m} v_y \quad (2)$$

Dans cette expression, g , K et m sont des constantes, mais v_y peut changer, donc l'accélération n'est pas constante !

Pour la suite, on pourra prendre les valeurs numériques suivantes :

- $m = 0,1 \text{ kg}$
- $g = 10 \text{ N/kg}$
- $K = 0,05 \text{ Ns/m}$

Question de compréhension 1 : On suppose que la bille part avec une vitesse initiale de module $v_i = 40 \text{ m/s}$. Que vaut a_y à ce moment là ?

.....

Question de compréhension 2 : Que vaut a_y lorsque la bille est au sommet de sa trajectoire ?

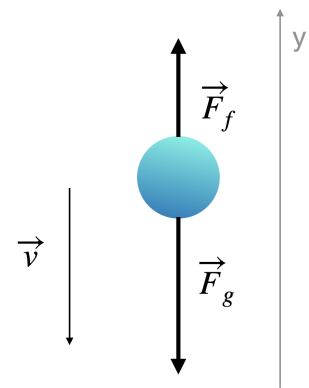
.....

Tir vertical (descente) et vitesse limite :

Lors de la descente, la vitesse change de sens, et donc le frottement fluide aussi. Les équations écrites précédemment reste vraie, mais v_y est maintenant négative :

$$a_y = -g - \frac{K}{m} v_y$$

Comme on l'a vu, le module de l'accélération valait g lorsque la bille était au sommet de la trajectoire. Mais lors de la descente, comme le frottement compense partiellement la gravité, le module de l'accélération devient progressivement plus petit que g . Si la bille chute suffisamment longtemps de cette manière, le module de l'accélération finit par devenir presque nul : la vitesse de la bille devient alors presque constante ! On dit que la bille atteint sa **vitesse limite**.



Question de compréhension 3 : Calculer la vitesse limite pour la bille considérée (si on suppose qu'elle retombe dans un puit et que le mouvement ne s'arrête pas quand elle revient à $y = 0 \text{ m}$).

.....

Calcul des valeurs pas-à-pas :

Avec une accélération qui dépend du temps, il va falloir calculer trois valeurs à chaque pas de temps : $y(t + \Delta t)$, $v_y(t + \Delta t)$ et $a_y(t + \Delta t)$.

La position en fonction du temps s'exprime comme dans le précédent chapitre du cours avec :

$$y(t + \Delta t) = v_y(t) \cdot \Delta t + y(t)$$

On obtient aussi la vitesse comme précédemment, sauf que l'accélération dépend du temps :

$$v_y(t + \Delta t) = a_y(t) \cdot \Delta t + v(t)$$

La nouvelle valeur de l'accélération est obtenue à partir de l'équation 2 :

$$a_y(t + \Delta t) = -g - \frac{K}{m} v_y(t)$$

2.2 Simulation

Exercices

Exercice 1:

1. Faire une copie du fichier de simulation que vous aviez obtenu à la fin du cours sur le MRUA. Donnez un nom qui évoque les frottements à cette copie (par exemple `frottement_fluide.py`). Nous allons travailler dessus.
2. On va faire au départ un lancer vertical, donc on va appeler nos positions y plutôt que x : c'est juste pour mieux comprendre si on doit relire le programme. Changez tout vos x en y (vous pouvez faire rechercher et remplacer).
3. Changez la valeur de l'accélération initiale à -9.8 . (En haut du document dans la partie Paramètre du mouvement) et mettez une vitesse initiale $v_0=10$.
4. Changez la valeur de dt pour qu'elle soit petite (de l'ordre de 0.01 pour éviter les erreurs de calcul).
5. Lancez votre programme et observez les courbes. Vous avez simulé un mouvement de chute libre à 1 Dimension !
6. Vous voyez que y peut devenir négatif, ce qui n'a pas de sens. On peut donc changer la condition du `while` : le mieux est probablement de se dire que l'on s'arrête dès que l'on touche le sol. Donc que la simulation continue tant que $y \geq 0$. Corrigez votre programme.

Exercice 2:

On va chercher à rajouter les frottements à notre simulation.

1. Dans les paramètres du mouvement, rajouter 2 variables k et m . Vous pouvez prendre $k=0.05$ et $m=0.1$ pour commencer.
2. Changez le nom de l'accélération initiale en a_0 (dans les paramètres).
3. À l'emplacement des condition initiales, mettez $a = a_0$
4. Rajoutez dans la boucle `while` la ligne qui permet de calculer l'accélération à chaque itération (attention, nous venons de définir a_0 comme étant égale à $-g$).
5. Supprimez la courbe `position2`, qui ne sert plus à rien.
6. Lancez votre programme, observez les courbes.
7. Vous avez simulé un mouvement de chute libre à 1 dimension avec frottement !

3 Simuler le mouvement d'un objet avec frottement fluide en 2 dimensions

3.1 Théorie

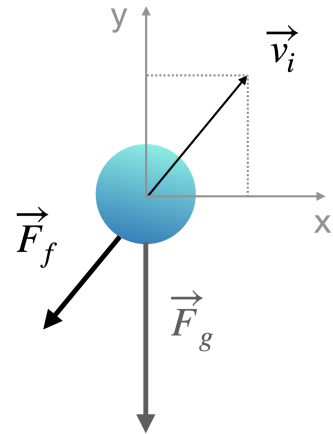
Tir oblique (mouvement en deux dimensions) :

On considère maintenant une bille qui est tirée avec une vitesse initiale oblique.

On note $\vec{v}$ la vitesse de la bille à un instant t donné, avec v_x et v_y les coordonnées horizontale et verticale de la vitesse.

Nous obtenons l'accélération avec la deuxième loi de Newton appliquée à la bille :

$$m\vec{a} = \vec{F}_g + \vec{F}_f$$



Mais cette fois-ci, il y a des forces selon l'axe x et selon l'axe y , donc on obtient les coordonnées a_x et a_y de l'accélération en décomposant la loi de Newton selon les deux axes :

$$\begin{cases} ma_x = -Kv_x \\ ma_y = -mg - Kv_y \end{cases}$$

En simplifiant par m , on obtient les deux coordonnées de l'accélération :

$$\begin{cases} a_x = -\frac{K}{m}v_x \\ a_y = -g - \frac{K}{m}v_y \end{cases} \quad (3)$$

On remarque encore une fois que les deux coordonnées de l'accélération dépendent de la vitesse de la bille et donc, a priori, du temps.

Question de compréhension 4 : On suppose que la vitesse initiale de la bille se décompose sur les axes x et y de la façon suivante :

- $v_{i,x} = 30\text{m/s}$
- $v_{i,y} = 40\text{m/s}$

Calculer les coordonnées de l'accélération initiale.

.....

.....

.....

Calcul des valeurs pas-à-pas :

En deux dimensions, six valeurs doivent être calculées à chaque pas de temps : $x(t + \Delta t)$ et $y(t + \Delta t)$, $v_x(t + \Delta t)$ et $v_y(t + \Delta t)$, et enfin $a_x(t + \Delta t)$ et $a_y(t + \Delta t)$.

Les positions en fonction du temps s'expriment avec :

$$x(t + \Delta t) = v_x(t) \cdot \Delta t + x(t) \quad \text{et} \quad y(t + \Delta t) = v_y(t) \cdot \Delta t + y(t)$$

Les vitesses sont obtenues avec :

$$v_x(t + \Delta t) = a_x(t) \cdot \Delta t + v_x(t) \quad \text{et} \quad v_y(t + \Delta t) = a_y(t) \cdot \Delta t + v_y(t)$$

Les nouvelles coordonnées de l'accélération sont obtenues avec les équations 3 :

$$a_x(t + \Delta t) = -\frac{K}{m}v_x(t) \quad \text{et} \quad a_y(t + \Delta t) = -g - \frac{K}{m}v_y(t)$$

3.2 Simulation

On va compléter le programme pour avoir un mouvement à 2 dimensions.

Exercices

Exercice 3:

1. Changez le nom des variables : v , a , position, vitesse, v_0 et a_0 en leurs ajoutant $_y$ au bout. Par exemple, v devient v_y , position devient position $_y$. Cela nous permettra d'identifier clairement les variables de notre axe vertical.
2. Pour chaque ligne, définition de variable ou calcul impliquant une variable avec y , copiez-la et changez les y en x .

Exemples :

- (a) pour $v0_y=10$, on ajoute juste en dessous $v0_x=10$
 - (b) pour $y=y+v_y*dt$, on ajoute juste en dessous $x=x+v_x*dt$
3. Mettez $a0_x$ à 0 ou bien supprimez-le du calcul de a_x .
 4. Nous allons maintenant tracer 6 graphiques. Ils apparaîtront sous la forme d'une grille 3×2 (3 lignes, 2 colonnes).
 - (a) Pour tous les graphiques existants, changez le `plt.subplot` pour que les 2 **premiers** chiffres soient 3 et 2.
 - (b) Copiez le code pour le graphique de la position et vitesse de y pour qu'ils utilisent les variables x . Donnez le numéro 3 et 4 aux graphiques (en changeant le dernier numéro de `plt.subplot`)
 - (c) Rajoutez 2 graphiques : un qui affiche la vitesse $_y$ en fonction de la vitesse $_x$ et un qui affiche la position $_y$ en fonction de la position $_x$. Donnez-leur les numéro 5 et 6. Assurez-vous d'avoir des titres corrects pour les axes.
 5. Lancez votre programme et vérifiez que vos graphiques vous semblent corrects.
 6. Changez les valeurs des paramètres initiaux pour voir leur impact.

Exercice 4:

Pour finir, nous allons créer l'animation :

1. Rajoutez `import matplotlib.animation as animation` tout en haut juste en dessous de l'autre importation. Elle permet d'importer les éléments nécessaires à l'animation.
2. Récupérez le fichier `animation.py` sur Moodle. Copiez l'ensemble du code présent et mettez-le à la fin de votre code.

3. Lancez votre code. Votre code devrait afficher les courbes et lorsque que vous quittez les courbes, l'animation devrait apparaître.
4. Jouez avec le paramètre `interval`.